MedConnect: Clinic Appointment Management System

Mid-Semester Database Design & Architecture Submission

---

1. Problem Statement & Design Assumptions

Objective: To design and implement a robust Relational Database Management System (RDBMS) that streamlines clinic operations, including patient registration, doctor scheduling, appointment concurrency, medical record management, and automated invoice generation.

Core Business Rules:

1. A doctor must not be double-booked for overlapping time slots at any given day.
2. Prescribing medicines must atomically decrement the pharmacy stock to maintain accurate inventory.
3. An invoice can only be generated for an appointment that has been marked `Completed`.

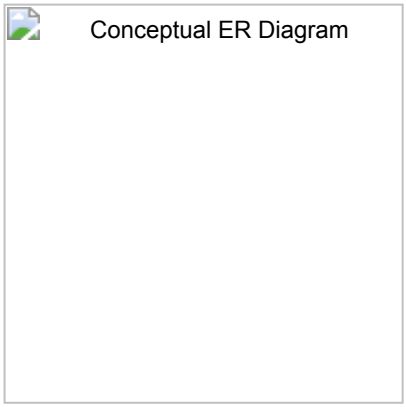
Design Assumptions:

1. A doctor belongs to exactly one department at any given time.
2. Each appointment generates at most one medical record and one consolidated invoice.
3. Cancelled appointment slots are retained in the database for audit history and may be re-booked.
4. The system supports two user roles: **Doctor** and **Admin/Receptionist**. Patients are managed as data entries, not system users, in this phase.

2. Conceptual Database Design (ER Diagram)

The conceptual model maps business logic using **Chen Notation**:

- **Regular Entities:** PATIENT, DOCTOR, DEPARTMENT, MEDICINE, APPOINTMENT.
- **Weak Entities (Double Borders):** DOCTOR_SCHEDULE, MEDICAL_RECORD, PRESCRIPTION, INVOICE.
- **M:N Relationships (Decomposed):** CONTAINS (Prescription–Medicine) → generates `Prescription_Items` table.
- **Identifying Relationships (Double Diamond):** GENERATES, HAS RX, HAS HISTORY, HAS SCHEDULE.

(Insert Conceptual ER Diagram) 

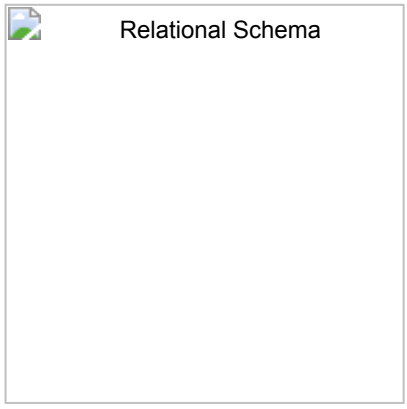
3. Logical Database Design (Relational Schema)

The conceptual model translates into a **10-table relational schema**. M:N relationships are decomposed into associative tables.

Tables with Primary Keys (PK) and Foreign Keys (FK):

#	Table	PK	FK(s)
1	`Departments`	dept_id	—
2	`Doctors`	doctor_id	dept_id → Departments
3	`Doctor_Schedules`	schedule_id	doctor_id → Doctors
4	`Patients`	patient_id	—
5	`Appointments`	appointment_id	patient_id → Patients, doctor_id → Doctors
6	`Medical_Records`	record_id	appointment_id → Appointments
7	`Prescriptions`	prescription_id	appointment_id → Appointments
8	`Medicines`	medicine_id	—
9	`Prescription_Items`	item_id	prescription_id → Prescriptions, medicine_id → Medicines
10	`Invoices`	invoice_id	appointment_id → Appointments, patient_id → Patients

(Insert Relational Schema Diagram)



4. Integrity Constraints & Normalization Justification

Important Integrity Constraints

- **Referential Integrity:** 15 Foreign Keys enforcing `ON DELETE RESTRICT`, preventing deletion of parent records that have active clinical dependencies.
- **Domain Constraints (24 CHECK constraints):**
  - `CHECK (payment_status IN ('Pending', 'Paid', 'Refunded', 'Cancelled'))`
  - `CHECK (consultation_fee >= 0.00)` — prevents negative billing.
  - `CHECK (total_amount = consultation_fee + medicine_total)` — enforces invoice balance.
- **Concurrency Constraint:** A partial unique index `uq_doctor_slot_active` prevents double-booking while allowing cancelled slots to be re-used.
- **UNIQUE constraints:** Doctor/Patient `phone` and `email` are globally unique across the system.

Normalization Justification (BCNF)

The schema is proven to be in **Boyce-Codd Normal Form (BCNF)**:

- **1NF:** All attributes are atomic (no repeating groups or arrays stored in single columns).
- **2NF:** No partial dependencies. In `Prescription_Items (prescription_id, medicine_id → quantity)`, `quantity` depends on the full composite key, not a subset.
- **3NF:** No transitive dependencies. `dept_name` is only stored in `Departments`, not repeated in `Doctors`.
- **BCNF:** For every non-trivial functional dependency $X \rightarrow Y$ across all 10 tables, X is a superkey.

5. Application Design & Team Contributions

Proposed Technology Stack

- **Database:** PostgreSQL 16.
- **Backend:** FastAPI (Python) — REST API handling concurrent requests and raw SQL transactions.
- **Frontend:** React.js + Tailwind CSS — clinical dashboard for staff and doctors.

High-Level Application Architecture

A 3-tier architecture: the **React frontend** communicates with the **FastAPI backend** via REST, which in turn executes raw parameterized SQL queries on the **PostgreSQL database**. Raw SQL (over ORM) is chosen to directly demonstrate DBMS features.

Proposed Workflows (Post Mid-Sem)

1. **Concurrent Booking Workflow:** An ACID transaction books a slot using a partial unique index to prevent race conditions.
2. **Clinical Encounter & Pharmacy Workflow:** A doctor issues a prescription; a PL/pgSQL trigger atomically decrements the pharmacy stock.
3. **Automated Billing Workflow:** A stored procedure aggregates consultation and pharmacy costs into a final invoice.

Mid-Sem Team Contributions

Member	Role	Mid-Sem Work Completed
Member 1	Database Architect	Drew the ER/EER conceptual diagram, identified all entities, relationships, and attributes.
Member 2	Schema & Keys Lead	Mapped the relational schema, assigned all PKs/FKs, and documented referential integrity rules.
Member 3	Constraints & Normalization	Wrote the 24 CHECK constraints, documented design assumptions, and proved BCNF compliance.
Member 4	App Design & Architecture	Defined the 3-tier tech stack, proposed the 3 clinical workflows, and set up the GitHub repository.
Member 5 *(if applicable)*	Documentation & Coordination	Authored the decision log (`decision.md`), coordinated contributions, and drafted this design document.